Projet de Programmation Distribuée

Task Manager — Architecture microservices sur Kubernetes

Master 1 Informatique

Année universitaire 2025-2026

Binôme	ADAMCZYK Nicolas / DINU Virgil
Encadrant	Benoît Charroux
Dépôt Git	https://github.com/virgil-dinu/prog_distribuee
Date de remise	26 avril 2026

1. Introduction

Dans le cadre de l'UE de programmation distribuée, nous avons choisi de développer une application de gestion de tâches (Task Manager) sous forme de microservices, afin d'intégrer l'ensemble des technologies imposées : conteneurisation Docker, orchestration Kubernetes, communication REST entre services, base de données persistante et gateway d'entrée.

L'objectif pédagogique de ce projet était pour nous de manipuler concrètement les briques d'une architecture distribuée moderne, plutôt que de développer une application fonctionnellement complexe. Nous avons donc délibérément choisi un domaine métier simple (CRUD de tâches + notifications) pour pouvoir nous concentrer sur l'intégration des technologies.

Le projet a été réalisé en binôme. Le code source est disponible sur notre dépôt Git mentionné sur la page de garde.

2. Choix techniques

2.1. Langage et framework

Nous avons retenu Node.js avec le framework Express pour l'implémentation des microservices. Ce choix est motivé par plusieurs raisons :

- Prise en main rapide, syntaxe familière pour un binôme de M1
- Écosystème npm riche (modules pg pour PostgreSQL, axios pour les appels REST inter-services)
- Images Docker Node Alpine très légères, ce qui accélère les cycles build/push/pull sur Minikube
- Démarrage rapide des conteneurs, utile lors des tests répétés

2.2. Communication inter-services

Nous avons implémenté la communication entre services en REST sur HTTP, avec un format JSON. L'option gRPC mentionnée dans le sujet aurait constitué un bonus, mais nous avons préféré sécuriser d'abord le socle REST fonctionnel plutôt que d'ajouter une complexité supplémentaire.

2.3. Base de données

PostgreSQL a été retenu plutôt que MySQL car il est largement utilisé en environnement Kubernetes et son image officielle Alpine est légère. La base est déployée dans le cluster via un Deployment dédié, avec un Service ClusterIP permettant aux autres services de la joindre par son nom DNS interne.

2.4. Frontend

Nous avons opté pour un frontend simple en HTML/CSS/JavaScript vanilla, servi par un conteneur nginx. Un framework type React aurait été surdimensionné pour l'usage qu'on en fait, et aurait

alourdi inutilement le projet. Le frontend appelle l'API via l'Ingress, sur les mêmes chemins que les services.

3. Architecture

Notre application est composée de quatre composants déployés dans un cluster Kubernetes local (Minikube) :

- frontend : conteneur nginx servant une page HTML/CSS/JS statique
- task-service : microservice Node.js/Express gérant le CRUD des tâches, persistées dans PostgreSQL
- notification-service : microservice Node.js/Express recevant les événements émis par task-service
- postgres : base de données relationnelle pour la persistance des tâches

Le schéma global du flux de requêtes est le suivant : l'utilisateur ouvre le frontend via l'Ingress, qui joue le rôle de gateway unique. L'Ingress route les requêtes vers le frontend (chemin /) ou vers les microservices (chemins /api/tasks et /api/notifications). Lors de la création ou suppression d'une tâche, task-service émet une notification REST vers notification-service.

```
distributed-project/
├── README.md
├── RAPPORT.md
├── task-service/             Node.js + Express + PostgreSQL, CRUD REST API
│   ├── server.js
│   ├── package.json
│   ├── Dockerfile
│   └── .dockerignore
├── notification-service/     Node.js + Express, receives events from task-service
│   ├── server.js
│   ├── package.json
│   ├── Dockerfile
│   └── .dockerignore
├── frontend/                 HTML + CSS + JS served by nginx
│   ├── index.html
│   └── Dockerfile
└── k8s/
    ├── 01-postgres-secret.yaml
    ├── 02-postgres.yaml
    ├── 03-rbac.yaml
    ├── 04-task-service.yaml
    ├── 05-notification-service.yaml
    ├── 06-frontend.yaml
    └── 07-ingress.yaml
```

3.1. Communication inter-services via DNS Kubernetes

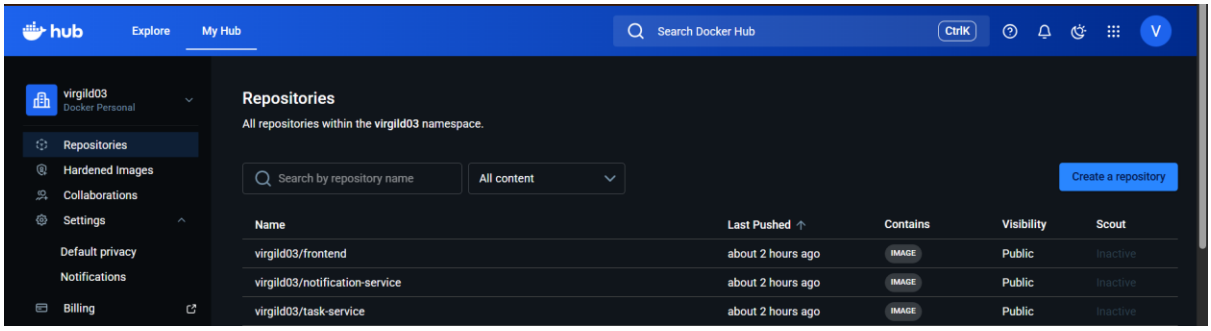
Chaque Service Kubernetes crée une entrée DNS interne au cluster. task-service peut ainsi appeler notification-service simplement via l'URL http://notification-service:4000, sans avoir à connaître l'adresse IP du pod cible. Le Service fait également office de load balancer lorsque plusieurs réplicas tournent.

4. Paliers de la grille d'évaluation

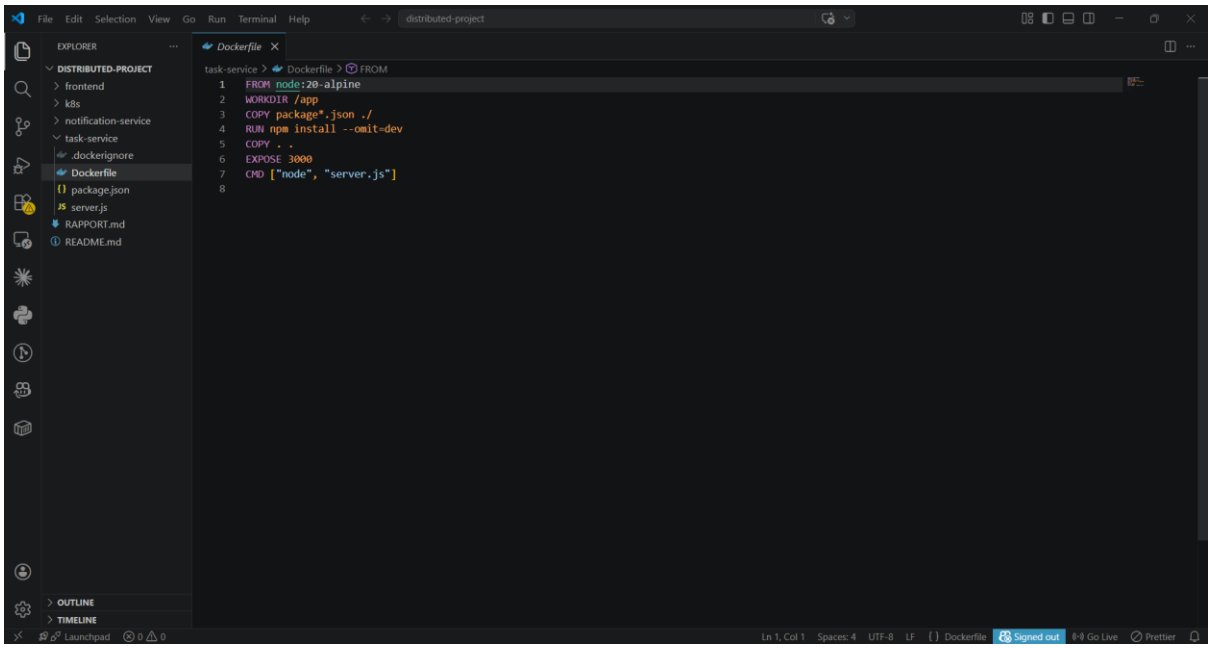
4.1. Palier 10/20 — Premier service en local

Nous avons codé le task-service en Node.js avec Express. Il expose une API REST à 4 endpoints : GET /api/tasks (liste), GET /api/tasks/:id (détail), POST /api/tasks (création), DELETE /api/tasks/:id (suppression). Un endpoint /health a également été ajouté pour les sondes Kubernetes.

Le service a été conteneurisé avec un Dockerfile basé sur node:20-alpine. L'image a été poussée sur Docker Hub, puis déployée dans Minikube via un Deployment (2 réplicas) et un Service ClusterIP.




```
C:\Users\lakaf>kubectl get pods
NAME                                  READY   STATUS    RESTARTS      AGE
frontend-5bf4b7cdb8-4f7w5             1/1     Running   0             103m
notification-service-676cd96d76-crlg6 1/1     Running   3 (31m ago)   103m
notification-service-676cd96d76-sccdr 1/1     Running   2 (31m ago)   103m
postgres-94c4b457c-bbwqn              1/1     Running   0             103m
task-service-5bb568f85-2bf8q          1/1     Running   4 (15s ago)   103m
task-service-5bb568f85-rch7m          1/1     Running   4 (15s ago)   103m
```



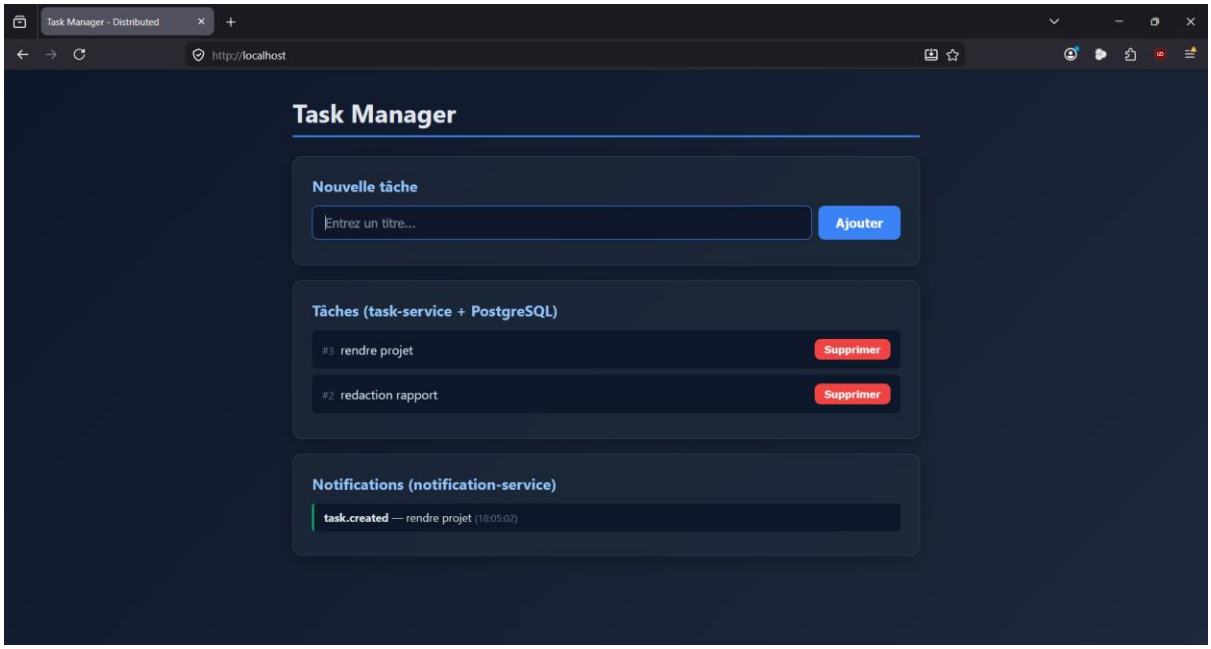
4.2. Palier 12/20 — Ajout d'une gateway

Nous avons activé l'addon ingress de Minikube, ce qui déploie un contrôleur NGINX Ingress dans le cluster. Un manifest Ingress a ensuite été créé pour router les requêtes entrantes :

- Le chemin / est routé vers le Service frontend
- Le chemin /api/tasks est routé vers le Service task-service sur le port 3000
- Le chemin /api/notifications est routé vers le Service notification-service sur le port 4000

Cette gateway unique simplifie beaucoup l'accès à l'application : une seule adresse, http://localhost/, expose l'ensemble des services. Cela permet également d'éviter les problèmes de CORS côté navigateur, puisque le frontend et les APIs partagent la même origine.

```
C:\Users\lakaf>kubectl get ingress
NAME           CLASS   HOSTS   ADDRESS        PORTS   AGE
main-ingress   nginx   *       192.168.49.2   80      107m
```



4.3. Palier 14/20 — Deuxième service relié au premier

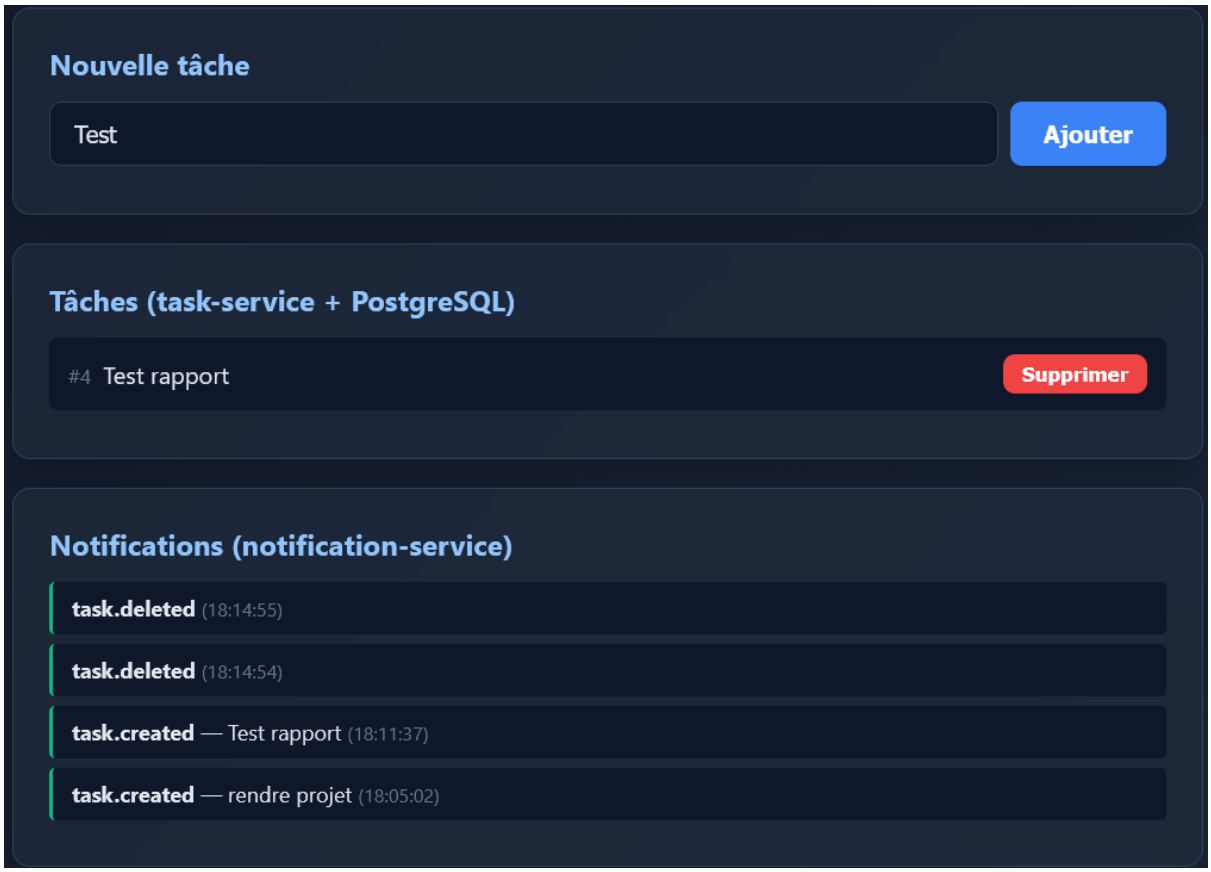
Nous avons ajouté un second microservice, notification-service, qui reçoit les événements métier émis par task-service. À chaque création ou suppression de tâche, task-service envoie une requête POST vers http://notification-service:4000/api/notifications avec la charge utile décrivant l'événement.

notification-service stocke les notifications reçues dans une liste en mémoire (plafonnée à 200 entrées) et les expose en lecture via GET /api/notifications. Le frontend interroge cet endpoint toutes les 5 secondes pour afficher un flux temps-réel.

Point d'attention rencontré : nous avons initialement oublié de déclarer le Service Kubernetes notification-service, ce qui faisait échouer la résolution DNS dans task-service. Une fois le manifest corrigé, l'appel fonctionnait immédiatement.

```
C:\Users\lakaf>kubectl logs -l app=task-service --tail=20
task-service listening on 3000
DB initialized
task-service listening on 3000
DB initialized
```

```
C:\Users\lakaf>kubectl logs -l app=notification-service --tail=20
notification-service listening on 4000
Notification reçue: {
  event: 'task.created',
  taskId: 3,
  title: 'rendre projet',
  receivedAt: '2026-04-20T16:05:02.039Z'
}
Notification reçue: {
  event: 'task.created',
  taskId: 4,
  title: 'Test rapport',
  receivedAt: '2026-04-20T16:11:37.364Z'
}
notification-service listening on 4000
```



4.4. Palier 16/20 — Base de données dans le cluster

Nous avons déployé PostgreSQL directement dans le cluster Kubernetes, via l'image officielle postgres:16-alpine. Le Deployment monte un volume de type emptyDir pour la simplicité (les données sont donc volatiles au redémarrage du pod, ce qui est acceptable pour un projet pédagogique ; en production un PersistentVolumeClaim serait nécessaire).

Les identifiants de connexion (utilisateur, mot de passe, nom de base) sont stockés dans un objet Secret Kubernetes, et injectés dans le pod postgres via envFrom ainsi que dans le pod task-service via valueFrom.secretKeyRef. Aucun mot de passe n'apparaît en clair dans les manifests commités.

Côté task-service, nous utilisons le module pg de Node.js avec un Pool de connexions. Au démarrage, le service crée la table tasks si elle n'existe pas, avec une boucle de retry de 15 tentatives espacées de 3 secondes pour tolérer le délai de démarrage de PostgreSQL.

```
postgres-94c4b457c-bbwqn                1/1     Running             0          71s
```

```
C:\Users\lakaf>kubectl get secret postgres-secret -o yaml
apiVersion: v1
data:
  POSTGRES_DB: dGFza3NkYg==
  POSTGRES_PASSWORD: dGFza3Bhc3MxMjM=
  POSTGRES_USER: dGFza3VzZXI=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"postgres-secret","namespace":"default"},"s
tringData":{"POSTGRES_DB":"tasksdb","POSTGRES_PASSWORD":"taskpass123","POSTGRES_USER":"taskuser"},"type":"Opaque"}
  creationTimestamp: "2026-04-20T14:16:39Z"
  name: postgres-secret
  namespace: default
  resourceVersion: "3912"
  uid: 310595e4-7424-4060-8cc2-f1a69cfb1712
type: Opaque
```

```
tasksdb=# \dt
         List of relations
 Schema | Name  | Type  |  Owner
--------+-------+-------+----------
 public | tasks | table | taskuser
(1 row)

tasksdb=# SELECT * FROM tasks;
 id |       title       | status  |         created_at
----+-------------------+---------+----------------------------
  2 | redaction rapport | pending | 2026-04-20 14:58:27.225164
(1 row)

tasksdb=# |
```

4.5. Palier 18/20 — Sécurisation via RBAC (partiel)

Nous avons mis en place un début de sécurisation au niveau Kubernetes via RBAC (Role-Based Access Control). Pour chaque microservice applicatif, nous avons créé :

- Un ServiceAccount dédié (task-service-sa, notification-service-sa) associé aux Deployments
- Un Role restreignant les droits de ce compte sur les ressources du namespace
- Un RoleBinding associant le ServiceAccount au Role

Concrètement, task-service-sa n'a le droit que de lire les pods, services et configmaps du namespace, ainsi que le Secret postgres-secret spécifiquement (par resourceNames). notification-service-sa a des droits encore plus limités : lecture pods et services uniquement. Ceci respecte le principe de moindre privilège : si un conteneur est compromis, l'attaquant ne peut pas lister d'autres secrets, créer des pods ou modifier la configuration du cluster.

Nous avons également défini des resources.requests et resources.limits sur tous les conteneurs, ce qui protège le cluster contre un pod qui consommerait toute la RAM ou le CPU disponible (attaque par déni de service via voisin bruyant).

```
C:\Users\lakaf>kubectl get sa,role,rolebinding
NAME                                 AGE
serviceaccount/default               3h3m
serviceaccount/notification-service-sa   121m
serviceaccount/task-service-sa       121m

NAME                                                    CREATED AT
role.rbac.authorization.k8s.io/notification-service-role   2026-04-20T14:16:40Z
role.rbac.authorization.k8s.io/task-service-role           2026-04-20T14:16:40Z

NAME                                                                ROLE                            AGE
rolebinding.rbac.authorization.k8s.io/notification-service-rolebinding   Role/notification-service-role   121m
rolebinding.rbac.authorization.k8s.io/task-service-rolebinding           Role/task-service-role           121m
```

```
frontend-5bf4b7cdb8-4f7w5                1/1     Running             0          20s
notification-service-676cd96d76-crlg6    0/1     Running             0          40s
notification-service-676cd96d76-crlg6    1/1     Running             0          42s
task-service-5bb568f85-rch7m             0/1     Running             0          46s
task-service-5bb568f85-rch7m             1/1     Running             0          52s
postgres-94c4b457c-bbwqn                 1/1     Running             0          71s
notification-service-676cd96d76-sccdr    0/1     Running             0          70s
task-service-5bb568f85-2bf8q             0/1     Running             0          72s
notification-service-676cd96d76-sccdr    1/1     Running             0          76s
task-service-5bb568f85-2bf8q             1/1     Running             0          78s
```

5. Procédure de déploiement

Le projet se déploie sur n'importe quel poste disposant de Docker Desktop, Minikube et kubectl. La procédure complète est détaillée dans le fichier README.md à la racine du dépôt. Résumé :

1. Démarrer Minikube avec l'addon ingress activé
2. Builder et pousser les 3 images Docker sur le Docker Hub personnel
3. Remplacer le placeholder YOUR_DOCKERHUB_USERNAME dans les manifests
4. Appliquer les 7 manifests numérotés via kubectl apply -f k8s/
5. Lancer minikube tunnel pour exposer l'Ingress sur localhost
6. Ouvrir http://localhost/ dans un navigateur

Les manifests sont numérotés de 01 à 07 pour garantir un ordre d'application cohérent : Secret en premier, puis PostgreSQL, puis RBAC, puis les microservices, puis l'Ingress.

```
frontend-5bf4b7cdb8-4f7w5                 1/1     Running             0          20s
notification-service-676cd96d76-crlg6     0/1     Running             0          40s
notification-service-676cd96d76-crlg6     1/1     Running             0          42s
task-service-5bb568f85-rch7m              0/1     Running             0          46s
task-service-5bb568f85-rch7m              1/1     Running             0          52s
postgres-94c4b457c-bbwqn                  1/1     Running             0          71s
notification-service-676cd96d76-sccdr     0/1     Running             0          70s
task-service-5bb568f85-2bf8q              0/1     Running             0          72s
notification-service-676cd96d76-sccdr     1/1     Running             0          76s
task-service-5bb568f85-2bf8q              1/1     Running             0          78s
```

6. Structure du dépôt

```
distributed-project/
├── README.md
├── RAPPORT.md
├── task-service/             Node.js + Express + PostgreSQL, CRUD REST API
|   ├── server.js
|   ├── package.json
|   ├── Dockerfile
|   └── .dockerignore
├── notification-service/     Node.js + Express, receives events from task-service
|   ├── server.js
|   ├── package.json
|   ├── Dockerfile
|   └── .dockerignore
├── frontend/                 HTML + CSS + JS served by nginx
|   ├── index.html
|   └── Dockerfile
└── k8s/
    ├── 01-postgres-secret.yaml
    ├── 02-postgres.yaml
    ├── 03-rbac.yaml
    ├── 04-task-service.yaml
    ├── 05-notification-service.yaml
    ├── 06-frontend.yaml
    └── 07-ingress.yaml
```

7. Difficultés rencontrées et résolutions

7.1. Synchronisation du démarrage task-service / PostgreSQL

Lors des premiers déploiements, task-service crashait immédiatement car PostgreSQL n'était pas encore prêt à accepter des connexions. Nous avons résolu ce problème en implémentant une boucle de retry dans la fonction initDB() du task-service : 15 tentatives espacées de 3 secondes. En pratique, la 2e ou 3e tentative réussit toujours, et le pod est déclaré ready via sa readinessProbe HTTP sur /health.

7.2. Accès à l'Ingress sur Minikube

L'activation de l'addon ingress ne suffit pas à rendre l'application accessible depuis le navigateur de la machine hôte. Il faut également lancer minikube tunnel dans un terminal séparé, qui demande le mot de passe administrateur pour créer un tunnel réseau vers le cluster. Une alternative est d'utiliser l'IP retournée par minikube ip, mais le tunnel est plus pratique pour le développement.

7.3. Propagation des images Docker

Au début, nous avons buildé les images localement sans les pousser sur Docker Hub, en espérant que Minikube les trouverait. Cela échouait avec une erreur ImagePullBackOff. Nous avons compris que par défaut, kubelet cherche les images sur le registre distant. Deux solutions sont possibles : pousser sur Docker Hub (ce que nous avons fait), ou configurer le shell pour utiliser le daemon Docker de Minikube via eval $(minikube docker-env).

8. Bilan

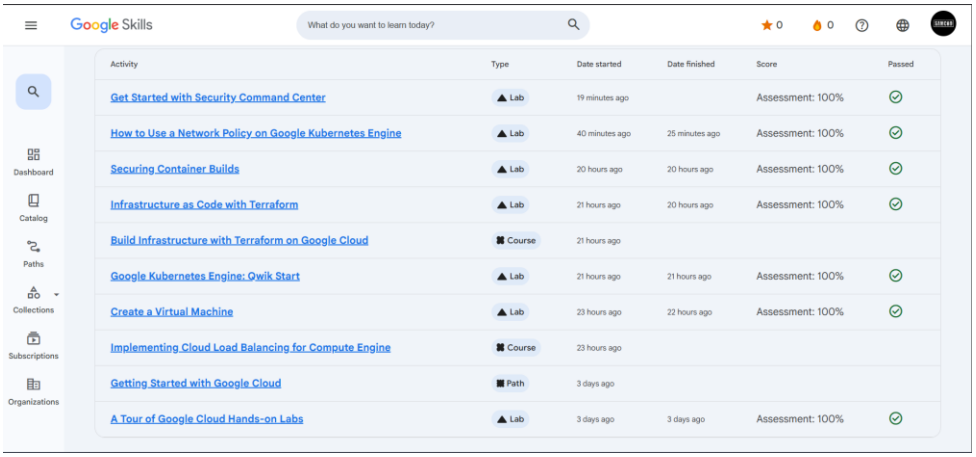
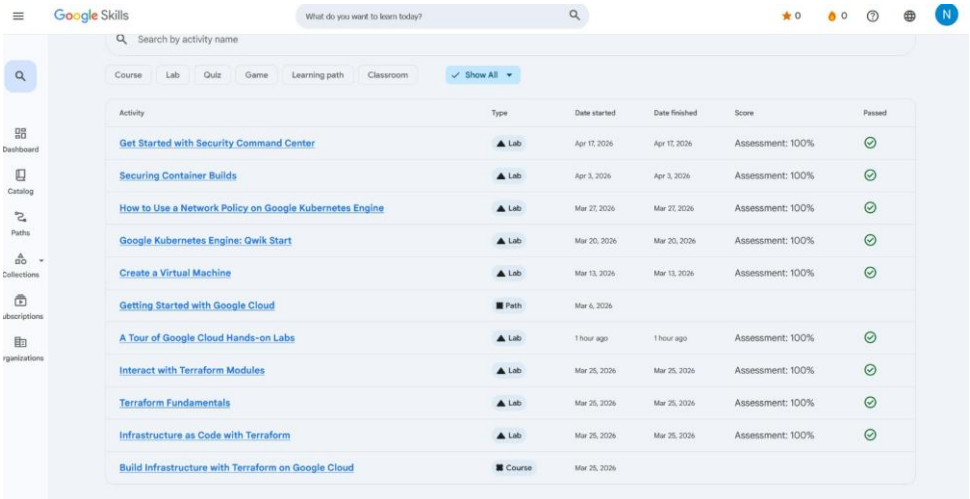
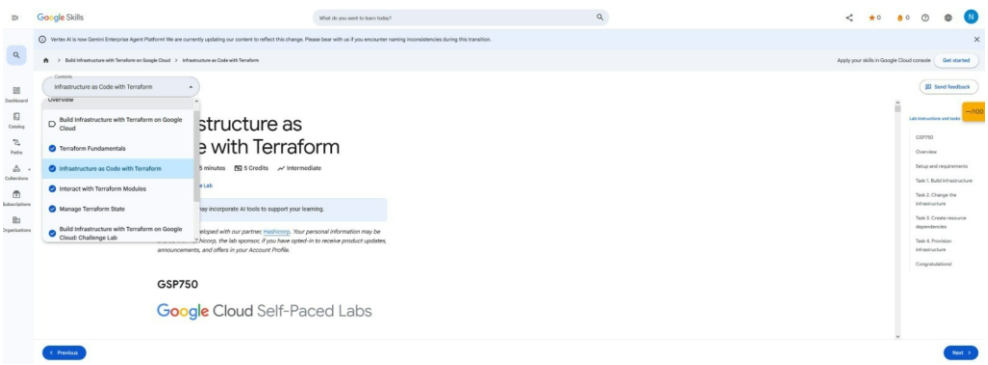
Ce projet nous a permis de manipuler concrètement l'ensemble de la chaîne de déploiement d'une application distribuée moderne. Les concepts vus en cours (Deployments, Services, Ingress, Secrets, RBAC) prennent tout leur sens dès lors qu'on les enchaîne dans un cas réel. Nous avons particulièrement apprécié la facilité avec laquelle Kubernetes permet de déclarer une architecture complexe de façon reproductible : n'importe quel membre du binôme pouvait reconstruire intégralement l'environnement avec kubectl apply -f k8s/ en quelques minutes.

Les axes d'amélioration identifiés pour une version ultérieure sont : migration vers un PersistentVolumeClaim pour la persistance réelle de PostgreSQL, mise en place de mTLS via un service mesh (Istio ou Linkerd), ajout de tests d'intégration automatisés, et déploiement sur un cloud public (GKE ou EKS) pour atteindre le palier bonus.

9. Activités Google Labs

Conformément aux consignes, nous joignons ci-dessous les captures d'écran individuelles de nos profils Google Labs.

9.1. DINU Virgil



9.2. ADAMCZYK Nicolas